# Algorithms and Programming

NAVIGAZIONE QUIZ

1 2 3 4

Successivo

Tempo rimasto **2:19:51**

Avvia una nuova anteprima

NAVIGAZIONE

Dashboard
- Home del sito
- Pagine del sito
- Corso in uso
- I miei corsi

AMMINISTRAZIONE
- Gestione Quiz
- Amministrazione del corso
- Cambia ruolo in ...

## Puoi visualizzare l'anteprima del quiz, tuttavia se questo fosse un tentativo reale, non ti sarà possibile in quanto:

Spiacente, il quiz non è disponibile

**Domanda 1**

Risposta non ancora data

Punteggio max.: 1,00

Contrassegna domanda

Modifica domanda

The following recursive function is called with the string s storing **"This is my string !"**

```
int rec (char *s) {
  if (*s=='\0')
    return 0;
  else
    return (((*s==' ') ? 1 : 0) + rec(s+1));
}
```

Indicate the exact value returned by the function.

Please, report a single integer value. No other symbols must be included in the response. This is an example of the response: 0

Risposta:

**Domanda 2**

Risposta non ancora data

Punteggio max.: 1,00

Contrassegna domanda

Modifica domanda

The following recursive function is called with n=4.

```
int rec (int n) {
  int i, res;
  if (n==0)
    return (1);
  res = 0;
  for (i=0; i<n; i++)
    res += rec(i) * rec(n - 1 - i);
  return res;
}
```

Following the recursion tree, report the exact value returned by the function to the initial caller.

Please, report the exact output displayed by the function. No other symbols must be included in the response. This is an example of the response: 0

Risposta:

**Domanda 3**

Risposta non ancora data

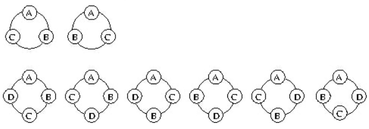
Punteggio max.: 6,00

Contrassegna domanda

Modifica domanda

The number of ways to arrange n distinct objects along a straight line is given by the **permutation**. The number of permutation of n objects is n!.
The number of ways to arrange n distinct objects along a circular line is given by the **circular permutation**. The number of circular permutations is (n-1)!. The number is (n-1)! since all cyclic permutations of objects are equivalent because the circle can be rotated.

For example, of the 3!=6 permutations of three objects {A,B,C}, the (3-1)!=2 distinct circular permutations are {A,B,C} and {A,C,B} as represented in the top section of the following figure. The other permutations, i.e., {B,C,A}, {C,A,B}, {C,B,A}, and {B,A,C}, are equivalent because they cannot be distinguished by the previous two when placed around a circle,
Similarly, of the 4!=24 permutations of four objects, only (4-1)!=6 are distinct circular permutations as represented by the bottom part of the figure.



Write the function **circularPermutation**, whose prototype is the following:

```
void circularPermutation (char *str);
```

which receives a string **str** and it displays all circular permutations of its characters.
Write the entire C code (no libraries are allowed).
Suggestion: Store all circular permutations in a proper data structure.

**Domanda 4**

Risposta non ancora data

Punteggio max.: 12,00

Contrassegna domanda

Modifica domanda

A WEB server stores all access on the WEB performed by all employees' computers. The format of the file is the following:

```
178.1.192.32 goodguy [11/01/2021:15:21:43] POST /intranet/login 200
178.1.192.42 nastyguy [10/01/2021:18:56:01] POST /intranet/login 200
178.1.192.33 goodguy [10/01/2021:13:55:36] GET index.html 200
34.52.1.33 badguy [10/01/2021:14:04:10] POST /services/ask 200
34.52.177.48 okguy [10/01/2021:14:32:00] POST /intranet/login 401
178.1.192.41 nastyguy [10/01/2021:18:29:01] POST /intranet/login 401
34.52.1.33 badguy [10/01/2021:14:06:20] POST /intranet/login 401
123.154.48.1 worstguy [11/01/2021:00:21:32] GET /services/list 200
```

Where the different fields report:

- The IP address of the host performing the WEB access
- The user name, the data, and time of the access,
- The type of access (GET or POST)
- The name of the resource searched by the host: 200 when the request was accepted and 401 when it was not accepted.

Suppose a reasonable format and a maximum length for each one of these fields. IP addresses and user names are unique within the database.
The candidate must write a C program able to store all information contained in the file in a data structure suitable to print all accesses performed by an IP address or a user name using as a search key either the IP address or the user name. More specifically, given an IP address **or** a user name, the data structure must guarantees searches with **logarithmic time costs**, in the average case, using either the IP address **or** the user name as a key. Moreover, once the entry of the IP address or the user name has been found, displaying the list of accesses performed by this IP or user must have a linear cost in the number of WEB accesses.

For example, given the previous file and the IP 34.52.1.33 the search must return the name of the user, i.e., badguy, and all information about his/her 2 accesses to the WEB, i.e., the ones of the [10/01/2021:14:04:10] and the one of the [10/01/2021:14:06:20]. Similar information has to be displayed if the search key is the name, i.e., badguy.

The candidate must:

1. Specify (in English language) which is the data structure and the logic used to respect the constraints in terms of complexity. The candidate must motivate the answer.
2. Define in C code the ADTs required to satisfy these constraints.
3. Write in C code the function **read**, which receives the name of the file containing the information, stores it in the data structure, and returns the pointer to such a structure. Notice that the function **read** must make appropriate use of all required library functions.

Library functions do not have to be reported in the examination text (a comment will suffice). Nevertheless, they must be inserted in the final program to verify the **read** procedure. All other parts (e.g., searching for an IP address or a user name in the data structure) do not have to be implemented (neither during the exam nor during the off-line verification session).

Successivo